MiniMart — Claude Code Build Instructions

Mining Site Inventory & Purchasing System

How to use this document

This guide is written for use with **Claude Code** — the command-line agentic coding tool. Each module is described as a *goal*, not a step-by-step recipe. Give Claude Code the goal for a module, let it build, review the output, and move on. You already have the demo app to reference for expected behaviour.

How to work through this document
1. Open your terminal in the minimart/ project folder.
2. Run: claude (to start Claude Code)
3. Paste the goal for the current module into Claude Code.
4. Review the output — compare it to the demo app if unsure.
5. Run the server and test before moving to the next module.
6. Commit to git between modules so you can roll back safely.

Prerequisites

Before starting, make sure the following are installed on your machine:

Tool	Version	Install
Python	3.11+	`python.org`
Node.js	18+	`nodejs.org`
Tesseract OCR	5.x	`brew install tesseract  (Mac)` `apt install tesseract-ocr  (Linux)`
Claude Code	Latest	`npm install -g @anthropic-ai/claude-code`
Git	Any	`git-scm.com`

Project setup

Your backend scaffold already exists from the demo phase. Open Claude Code from the project root:

```
cd minimart/ claude
```

Claude Code will read your existing files automatically. You don't need to explain what's already there — just give it the goal for the next module.

The 5 Modules

Work through modules in order. Each module depends on the one before it. Module 1 is already complete from the demo scaffold.

Module 01 Project Setup & Database Layer — COMPLETE

Goal for Claude Code

This module is already done. The database schema, SQLAlchemy models, Alembic migration config, and FastAPI app shell are all in place. Run python -m app.db.init_db to create the database and seed the default admin account (ADMIN001 / PIN 0000). Change the PIN after first login.

Expected outputs

- SQLite database created at minimart.db
- All 6 tables: workers, products, orders, order_items, invoices, invoice_items
- Default admin account: ADMIN001 / PIN 0000
- FastAPI server starts with: uvicorn app.main:app --reload

Notes

- The dummy data seed script (seed_demo.py) is included in the zip for demo purposes.
- For production, use Alembic migrations instead of create_all.

Module 02 Authentication — NEXT

Goal for Claude Code

Build a login system where workers identify themselves using their Employee ID and a 4-digit PIN. The system should return a JWT token that expires after 8 hours (one full shift). There are two roles: worker and admin. Protected routes should reject requests without a valid token, and admin-only routes should reject worker tokens. Workers should only be able to see their own data.

Expected outputs

- POST /api/auth/login → returns JWT token
- GET /api/auth/me → returns current user from token
- Auth dependency that any route can use to require login
- Role dependency that routes can use to require admin access
- Pydantic schemas for login request and token response

Notes

- PIN is stored as a bcrypt hash — never plaintext.
- JWT secret lives in .env — Claude Code should create a .env.example file.
- 8-hour expiry matches a typical site shift so workers don't need to re-login mid-shift.
- Test with: POST /api/auth/login body: {employee_id: 'ADMIN001', pin: '0000'}

Module 03 Invoice Parser (PDF / OCR) UPCOMING

Goal for Claude Code

Build an invoice upload and parsing system. An admin uploads a PDF invoice from a supplier. The system extracts line items (product name, quantity, unit price) using pdfplumber for digital PDFs and Tesseract OCR for scanned ones. The parsed results are saved with status pending_review — no inventory is changed yet. The admin then reviews each line item, maps it to the correct product in the database, and approves. On approval, stock levels are updated and the invoice is marked approved.

Expected outputs

- POST /api/invoices/upload → accepts PDF, runs parser, saves to DB
- GET /api/invoices/ → list all invoices with status
- GET /api/invoices/{id} → single invoice with all line items
- PUT /api/invoices/{id}/items/{item_id} → map raw OCR text to a product
- POST /api/invoices/{id}/approve → commit stock changes, mark approved
- POST /api/invoices/{id}/reject → discard without changing stock

Notes

- The hybrid approach: try pdfplumber first, fall back to Tesseract if no text is found.
- raw_product_name preserves the original OCR text — never overwrite it.
- product_id on invoice_items stays null until the admin confirms the mapping.
- Store uploaded PDFs in an uploads/ folder — add this to .gitignore.
- Reference the demo app Invoice Review screen for the expected admin workflow.

Module 04 Worker App API (React PWA backend) UPCOMING

Goal for Claude Code

Build the API endpoints that power the worker-facing mobile app. Workers should be able to browse all products that are in stock, add items to an order, and submit that order. Submitting an order decrements stock immediately and records the purchase against the worker's account for deduction tracking. Workers can also view their own order history and a running total of what will be deducted from their salary.

Expected outputs

- GET /api/products/ → list all products with stock > 0

- POST /api/orders/ → create a new order (worker auth required)
- GET /api/orders/my → worker's own orders and totals
- GET /api/orders/my/spending → total spend this month
- Stock is decremented atomically when order is created
- Order total is computed server-side from current prices (not client-submitted)

Notes

- Prices must be computed server-side to prevent tampering.
- If a product goes out of stock between browsing and checkout, return a clear error.
- The React PWA from the demo zip is the reference UI — connect it by changing useDummyData to false.
- Order status starts as pending — it becomes fulfilled when admin marks it done.

Module 05 Admin Dashboard API UPCOMING

Goal for Claude Code

Build the API endpoints for the admin panel. Admins need to manage inventory, fulfill or cancel worker orders, and generate spending reports for the payroll team. The spending report is the most critical output — it shows exactly how much to deduct from each worker's salary. It should be filterable by date range and exportable as a CSV file that accounting can use directly.

Expected outputs

- GET /api/admin/orders/ → all orders, filterable by status/date/worker
- PUT /api/admin/orders/{id}/fulfill → mark order fulfilled
- PUT /api/admin/orders/{id}/cancel → cancel an order
- GET /api/admin/products/ → full inventory with stock levels
- PUT /api/admin/products/{id} → update product price or stock
- POST /api/admin/products/ → add a new product
- GET /api/admin/reports/spending → per-worker totals, filterable by date
- GET /api/admin/reports/spending.csv → CSV export for payroll

Notes

- All admin routes must be protected by the admin role dependency from Module 2.
- The CSV export should include: worker name, employee ID, total deduction, month.
- Low stock warning threshold: flag products with stock <= 5 in the inventory endpoint.
- Reference the demo Admin Dashboard for the expected data shapes.

Connecting the demo app to the real backend

The React demo app is pre-wired to talk to the real backend. There is one change to make:

In minimart-demo/src/data/dummyData.js, change:

```
export const USE_DUMMY_DATA = true
```

to:

```
export const USE_DUMMY_DATA = false
```

Then set the API base URL to your running FastAPI server (default: http://localhost:8000). The app will switch from local dummy data to live API calls automatically. Run both servers side by side during development:

```
# Terminal 1 — FastAPI backend cd minimart/ uvicorn app.main:app --reload # Terminal 2 — React frontend cd minimart-demo/ npm run dev
```

Deployment checklist

When moving from development to production on the mining site server:

Environment

- Copy .env.example to .env and set a strong SECRET_KEY (32+ random chars)
- Set DATABASE_URL to PostgreSQL connection string
- Set ENV=production and DEBUG=False

Database

- Run: alembic upgrade head (runs all migrations cleanly)
- Do NOT use Base.metadata.create_all() in production — use Alembic only
- Run the seed script once to create the admin account

Backend

- Use gunicorn with uvicorn workers: gunicorn app.main:app -w 4 -k uvicorn.workers.UvicornWorker
- Put behind nginx as a reverse proxy
- Set CORS origins to your actual frontend domain, not wildcard *

Frontend

- Run: npm run build in minimart-demo/
- Serve the dist/ folder via nginx or upload to any static host
- Set the API base URL to the production backend domain

Security

- Change ADMIN001 default PIN immediately after first login
- Restrict /api/admin/* routes to your internal network if possible

- Enable HTTPS — workers will be sending PINs over the network

---